

Plume TTS Audio Engine

My Debugging Notes

What broke, what I tried, what failed, and how I finally fixed it

Project	Plume TTS Audio Engine
What I was working on	TTS generation and audio playback using MLX Swift and AVFoundation
Problems	Backgrounding the app caused crashes, and audio chunks had noticeable gaps
Time spent debugging	About 3–4 days
Why I'm keeping this	So I remember how I solved the problem and have a good example for future interviews

I'm keeping this as a record of the debugging process, not only the final fix. The failed attempts are important because they show how I figured out what was actually happening.

1. The Background Crash

What was happening

I ran into a crash whenever I put the app in the background while TTS was generating audio. The app would hit a SIGABRT almost immediately.

After tracing the crash, the issue came down to MLX Swift still submitting Metal GPU commands while the app was moving into the background. iOS does not allow those GPU operations once background execution starts.

What I tried first

Attempt	What I did	What went wrong
1. Async cancellation	I called <code>pauseEngine()</code> inside a Task when the app was about to go into the background.	The async task was too slow. iOS had already revoked GPU access before the pause code ran.
2. Pause + GPU sync	I set <code>isPaused = true</code> synchronously and called GPU synchronization.	This stopped new chunks from starting, but the current MLX C++ evaluation was still running. So GPU work was still being submitted after the synchronization call.

The part that finally clicked

I realized I was thinking about the problem as if “paused” meant “nothing is running.” It didn’t. The pause flag only stopped the next chunk. The current MLX evaluation was still doing work in the background.

So the real question became: how do I make sure the active MLX evaluation has completely finished before iOS takes away GPU access?

The fix

1. Stop new work

In `willResignActive`, I synchronously set `isPaused = true` on a thread-safe background state object. This prevents another text chunk from starting.

2. Wait for the current work

I then wait for the MLX background thread to finish its current evaluation.

3. Add a safety limit

I added a 3.5-second timeout so the app does not sit there forever and get killed by the iOS Watchdog.

4. Sync the GPU

Once the evaluation is finished, I call `MLX.Stream.defaultStream(.gpu).synchronize()` so the remaining Metal command buffers finish before the lifecycle callback returns.

5. Resume normally

When the app becomes active again, I set `isPaused = false` and let the generation loop continue.

2. The Audio Gaps

What was happening

After fixing the crash, I had another problem. The actual speech worked, but there was a noticeable little pause between sentences or paragraphs.

I traced this to two things.

Problem	Why it mattered
Separate AVPlayerItems	Every text chunk became its own WAV file and its own AVPlayerItem. Moving from one item to the next introduced decode and buffer latency.
Trailing silence	The MLX TTS engine added around 83 ms of silence to the end of each chunk to prevent clipping.

What I changed

Instead of treating every chunk as a separate player item, I changed the playback architecture to use one AVPlayer backed by an AVMutableComposition.

Now, when a new chunk finishes generating, I append the audio directly to the end of the existing timeline. The player gets an updated version of the composition, so the audio keeps extending without having to jump from one player item to another.

I also reduced the trailing silence from about 83 ms to about 20 ms. That was enough padding to prevent clipping while making the gap basically imperceptible.

What I ended up with

- One continuous audio timeline
- No noticeable transition between chunks
- Simpler time-based seeking
- No need for the old silence-sentinel workaround

3. What I Learned

The biggest lesson from the crash was that changing a state variable and stopping the work behind that state are two different things.

I also learned that the timing of lifecycle events matters a lot when GPU work is involved. A fix that looks correct at the application level still fails if a native operation is already running underneath.

For the audio problem, I learned that the playback architecture itself was causing part of the issue. Tweaking timing would not have been enough. I needed a continuous timeline instead of a chain of separate items.

4. How I'd Explain This in an Interview

I spent about three to four days debugging a couple of problems in my TTS audio engine. The first was an iOS crash whenever the app went into the background during generation. I traced it to MLX still submitting Metal GPU work after iOS revoked GPU access. My first two fixes failed because I was stopping new work without

making sure the work already running had finished. I ended up using cooperative cancellation, waiting for the active MLX evaluation to finish, adding a 3.5-second watchdog, and then synchronizing the GPU before returning from the lifecycle callback.

I also had noticeable gaps between generated audio chunks. I found that separate AVPlayerItems introduced transition latency, while the TTS engine was adding around 83 ms of trailing silence. I replaced the queue-based architecture with an AVMutableComposition timeline and reduced the padding to around 20 ms. That gave me continuous playback and a simpler seeking model.

5. Quick Reference

Area	Before	After
Background handling	Async pause request	Pause + wait for active MLX evaluation
GPU synchronization	Called before active evaluation was finished	Called after evaluation stopped
Watchdog protection	No timeout	3.5-second timeout
Audio playback	AVQueuePlayer + separate items	AVPlayer + AVMutableComposition
Trailing silence	~83 ms	~20 ms

Source: my original Plume TTS Audio Engine post-mortem notes.